Optimizing Virtual Channel Establishment in the Face of On-Path Adversaries

Abstract. Payment channel networks (PCNs) are among the most promising solutions to the scalability issues in permissionless blockchains, by allowing parties to pay each other off-chain through a path of payment channels (PCs). However, routing transactions comes at a cost which is proportional to the number of intermediaries, since each charges a fee for the routing service. Furthermore, analogous to other networks, malicious intermediaries in the payment path can lead to security and privacy threats. Virtual channels (VCs), i.e., bridges over PC paths, mitigate the above PCN issues, as an intermediary participates only once to set up the VC and is then excluded from every future VC transaction. However, similar to PCs, creating a VC has a cost that must be paid out of the bridged PCs' balance. Currently, we are missing guidelines to where and how many VCs to set up. Ideally, VCs should minimize transaction costs while mitigating security and privacy threats from on-path adversaries. In this work, we address for the first time the VC setup problem, formalizing it as an optimization problem. We present an integer linear program (ILP) to compute the optimal VC setup strategy in terms of transaction costs, security, and privacy. We also propose a greedy alternative to solving the computationally intensive ILP. Finally, we conduct an evaluation of our greedy approach over a snapshot of the Lightning Network (LN), the largest Bitcoin-based PCN, confirming on real-world data that our strategy minimizes costs while protecting against security and privacy threats of on-path adversaries. These findings may serve the LN community as guidelines for the deployment of VCs.

1 Introduction

Permissionless cryptocurrencies face severe scalability challenges, as they rely on a set of mutually untrusted users located across the world to maintain a distributed and publicly verifiable transaction ledger. The transaction throughput today is limited to tens of transactions per second at best, while each individual transaction can take up to 60 minutes to be confirmed.

Payment channels (PC) have emerged as one of the most promising scalability solutions, and instances such as the Lightning Network [30] are gaining traction. In this approach, Alice and Bob can create a PC between them with a single on-chain transaction that transfers their coins into an escrow (or multisignature) controlled by both of them with the additional guarantee that they can get refunded at a mutually agreed upon time. After that, Alice and Bob can pay each other off-chain by exchanging authenticated copies of the updated balances in the escrow. Finally, the PC is closed with an on-chain transaction representing the last authenticated distribution of coins.

PCs can be linked to form a network, also called payment channel network (PCN), where any two users can perform a payment if they are connected by a path of PCs. The payment in the PC between Alice and the first intermediary is forwarded along the intermediary PCs until it reaches Bob. A key challenge in this approach is then to ensure that the balance updates of all PCs in the path are atomic to prevent any intermediary (i.e., Ingrid) from trivially stealing the money by denying forwarding it.

State-of-the-art techniques to construct atomic multi-hop payments [2, 9, 15, 17, 22, 23, 36] require that intermediaries are involved in every single payment. This approach brings several disadvantages: (i) reduction of the payment reliability (e.g., Ingrid may simply be offline or crash); (ii) increase in the payment latency since additional PCs are required; (iii) high payment costs as each intermediary charges a fee per transaction for providing the routing service; and (iv) possible leakage of sensitive information to the intermediaries, which opens the door to a number of security and privacy issues, such as route hijacking [40], wormhole attacks [23] or user anonymity [22], just to name a few.

Recently, the concept of the virtual channel (VC) [1,3,8,16] has been proposed to improve upon the aforementioned drawbacks of PCs. A VC can be seen as a bridge over two PCs. For instance, assume that Alice and Bob have a PC with an intermediary, Ingrid. In order to set up a VC, Ingrid must collaborate and coordinate with Alice and Bob to lock coins in their corresponding PCs in order to use those coins to build a VC directly between Alice and Bob. This approach brings the following benefits: (i) Alice and Bob can pay each other "as if they would have a PC between them", that is, without the involvement of Ingrid; (ii) payment latency is reduced to one hop; (iii) payment fees charged by the intermediaries for their routing service are avoided[1]; and (iv) the details of every single payment are not revealed to possibly malicious or curious intermediaries.

A crucial question, not yet addressed in the literature, is *what strategy should users follow to open VCs while optimizing the cost-effectiveness, as well as on-path security and privacy benefits provided by VC networks?* This is an optimization problem, given that the funding to be locked, and thus the number of VCs a party can establish, is limited by the number of underlying PCs and the amount of coins that are locked on them. To investigate on-path security and privacy, we look at how three exemplary well-studied attacks in the literature can be prevented. Note that, while several existing works have studied from a game theoretic perspective how a PCN should evolve based on the fee optimization goal of the users [5, 6, 12], none of them considered virtual channels, nor security or privacy goals.

We make the following ***contributions***. First, we address the VC setup problem, formalizing it as an optimization problem of three distinct goals: (i) cost-effectiveness of the transactions (i.e., fees) while providing (ii) security and (iii) privacy guarantees against on-path adversaries and prove that the optimization problem is NP-hard. On-path adversaries account for a significant share of

[1] Note that intermediaries still charge fees for the coordination service when establishing the virtual channel.

attacks in the PCN-related literature: e.g., they may aim to perform denial-of-service and wormhole attacks, or to harm value privacy and relationship anonymity properties, among many others [11, 22, 23, 27, 37, 40, 41]. Such attacks have been shown to potentially have a severe impact in practice [38]. On-path adversaries can do damage depending on the attack that they are carrying out. In this work, we study three exemplary attacks and argue that a mitigation strategy subsumes protection against other on-path attacks as well.

Second, we analytically show a synergy between the different VC optimization objectives. In particular, we prove that minimizing transaction fees by the appropriate use of VCs also prevents attacks from on-path adversaries, such as those against value privacy and relationship anonymity, or wormhole attacks. In practice, this implies that users can set up their VCs following a single strategy to minimize their transaction costs, and as a side benefit, they will be secure against the three exemplary on-path attacks on security and privacy in study.

Third, and motivated by the uncovered synergy between the objectives, we describe concrete approaches to devise fee optimization strategies which mitigate three security and privacy attacks (value privacy, relationship anonymity and wormhole attacks). In particular, we present both a rigorous and exact approach (based on integer linear programming), which is computationally intractable, and an efficient approach (based on a greedy routing algorithm) to optimize the cost-effectiveness, security, and privacy of PCNs through the use of VCs.

Finally, we evaluate our optimization approach on a recent snapshot of the Lightning Network (LN). We show that our transaction cost minimization strategy is efficient and effective, and indeed subsumes the strategies to optimize for security and privacy. We find that depending on how many payments two endpoints plan to conduct via the virtual channel, the routing cost can be reduced significantly, for example, to about half compared to a normal payment for two consecutive payments, or to about 3% for 50 consecutive payments. In addition to this cost reduction, other users can utilize these virtual channels to route their payments through a potentially cheaper path.

To summarize, for the first time, we present both an analytical and an empirical study of the impact of using VCs in (current) PCNs in terms of cost-effectiveness of the transactions as well as security and privacy guarantees. The results of this work motivate the deployment of VCs and should prompt the PCN community and developers to include VCs within current PCNs software and make them accessible to the PCN users.

2 Background and problem overview

Payment channel networks (PCNs). A PCN [22] is a directed graph $\mathcal{G} := (\mathcal{V}, \mathcal{E})$. Nodes $\mathcal{V}$ represent users and edges $\{e_{i,j}, e_{j,i}\} \subset \mathcal{E}$ represent PCs between users. The weight on a directed edge denotes the amount of remaining coins that can be forwarded on that direction. For every pair of edges $\{e_{i,j}, e_{j,i}\}$, users v_i and v_j can exchange any part of their balance freely. Moreover, each directed edge $e_{i,j}$ between users v_i and v_j is associated with two non-negative

numbers, the base fee f_i, and the proportional fee p_i, that together determine the fees that each user charges for forwarding the payments. For a forwarded amount α via $e_{i,j}$, v_i charges $fee(e_{i,j}, v_i) = f_i + p_i \cdot \alpha$. We denote a payment channel $\{e_{i,j}, e_{j,i}\}$ with the tuple $(pc_{\langle v_i, v_j \rangle}, \beta_i, \beta_j, f_i, f_j, p_i, p_j)$, where β_k, $k \in \{i, j\}$ is the initial balance of each node upon channel creation.

The success of a payment between two users depends on the capacity available in the path connecting the sender s to the receiver r. Assume that s wants to pay α coins to r and that they are connected through a path $s \to u_1 \to \ldots \to u_n \to r$. The fees charged for every node in the path depend on the forwarded amount. That is, v_n charges $fee_n = f_{n,n+1} + p_{n,n+1} \cdot \alpha$ and in general v_j charges $fee_j = f_{j,j+1} + p_{j,j+1} \cdot (\alpha + \sum_{k=j+1}^{n} fee_k)$, for $j = 1, \ldots, n$ (each node forwards α and the forwarding fees of the remaining nodes in the path). Such a payment is successful if (i) s starts the payment with a value $\alpha^* := \alpha + \sum_{j=1}^{n} fee_j$ and (ii) every edge on the path has a balance of at least α'_i, where $\alpha'_i := \alpha^* - \sum_{j=1}^{i-1} fee_j$ (the initial payment value α minus the fees charged by the previous users in the path), $e_{j,j+1} = (u_j, u_{j+1})$, and $u_{n+1} = r$. If the payment is successful, the balance of every edge $e_{j,j+1}$ on the path from s to r is decreased by α'_i, while the balance of every edge $e_{j+1,j}$ is increased by α'_i.

PCN challenges. PCNs suffer from the following drawbacks:

Reliability: The success of a payment requires that each intermediary user must: (i) be online and (ii) follow the protocol honestly. For instance, in the example shown in Figure 1, the intermediate user (marked as red) could be offline or simply not forward the payment, making it fail.

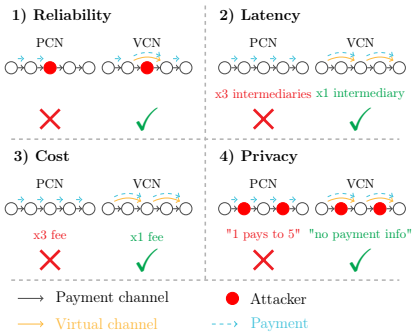

Fig. 1: Comparison between PCN and VCN.

Latency: The time to process a payment is directly proportional to the number of intermediate users. For instance, the latency of the payment shown in Figure 1 (latency section) could be reduced if a shorter path between nodes 1 and 5 would exist. However, creating a direct payment channel between nodes 1 and 5 in the PCN setting is expensive as the user 1 must own enough coins to lock them into a new payment channel, requiring also a new on-chain transaction.

Cost: The payment cost is proportional to the number of intermediate users, since each charges a routing fee. Similar to latency, it would be interesting to have shorter paths to reduce the cost, but it is an expensive operation.

Privacy: Each payment participant learns sensitive information. Recent work [13, 22] has shown that intermediaries can learn details about who pays what to whom in the currently deployed Lightning Network. While first alternative payment mechanisms are emerging that hide (some of) the information required in such payment, e.g., [2, 23], they have not been adopted yet and still protect

only some sensitive information but not other (e.g., the payment amount) and also do not decrease routing fees.

Virtual channels (VCs). A VC can be created between two users, say Alice and Bob, who have a PC with a common intermediary, say Ingrid. Using a 3-party protocol, the users can block coins in the underlying PCs and move them into the VC between Alice and Bob. After that, Alice and Bob can perform arbitrary many off-chain payments between them without involving Ingrid. The amount of virtual channels that can be created are thus limited by the balances available at the underlying PCs. Yet, it is interesting to deploy VCs as they provide several advantages over PCNs.

Reliability: Payments are carried out without involving the intermediary user, who cannot thus disturb it either actively (e.g., dropping it) or passively (e.g., being offline). In Figure 1, the malicious node 3 does not participate in the payment between 1 and 5 since it is omitted by the VC between 2 and 4.

Latency: The creation of virtual channels between users effectively creates shorter paths between possible sender and receiver pairs. Since there exist less intermediate users that need to process the payment, the latency of the overall payment is reduced. In the running example, the latency is reduced from 3 to 1 intermediaries, assuming that two virtual channels have been created.

Cost: Assume, for simplicity, that users charge the same fees for forwarding a payment through a payment channel and a virtual channel. In such a case, as with latency, the fact that virtual channels lead to shorter paths, can also help to reduce the overall payment cost in terms of fees. In Figure 1, the transaction cost using VCs is reduced to the fee charged by the only intermediary that is involved, avoiding thus the fees charged by nodes 2 and 4.

Privacy: The fact that some intermediate user is no longer participating in the payment improves the privacy of the overall payment. Note that although the intermediate user learns what are the two endpoints of the virtual channel since it was part of the 3-party protocol used to create it, the intermediate user no longer sees the amounts of the individual payments routed through the virtual channel. For instance, in Figure 1, the malicious node 2 would learn that there exists a VC between nodes 1 and 3 as it needs to help them to set the channel up, but afterwards the node 2 does not learn when a virtual channel is used.

VCs in practice. Despite the advantages provided by VCs, we currently lack a comprehensive analysis leading to a set of guidelines to help the users decide when to open virtual channels, with what neighbors and under what circumstances. Note that ideally, one user would like to open a virtual channel with every other user in the network. Unfortunately, this is not possible since each user has a limited budget, namely, the amount of coins available at her payment channels since they need to be locked to create a virtual channel. In this state of affairs, the following questions arise: how should a user choose which neighbor to open a virtual channel with? how many payments are required to amortize the cost of opening a VC? what strategy should a user follow to maximize the security and privacy gains against on-path adversaries while opening VCs?

$\underline{openPC\ (v_1, v_2, \beta_1, \beta_2, f_1, f_2, p_1, p_2)}$

$v_{i\in\{1,2\}}$: Nodes
$\beta_{i\in\{1,2\}}$: PC initial capacity
$f_{i\in\{1,2\}}$: Base routing fees
$p_{i\in\{1,2\}}$: Proportional routing fee

– If $\mathbb{B}[v_1] < \beta_1$ or $\mathbb{B}[v_2] < \beta_2$, abort
– Create a new payment channel $(pc_{\langle v_1,v_2\rangle}, \beta_1, \beta_2, f_1, f_2, p_1, p_2) \in \mathcal{E}_p$
– Update the blockchain as $\mathbb{B}[v_1] = \mathbb{B}[v_1] - \beta_1$ and $\mathbb{B}[v_2] = \mathbb{B}[v_2] - \beta_2$

$\underline{closePC\ (pc_{\langle v_1,v_2\rangle})}$

$pc_{\langle v_1,v_2\rangle}$: PC identifier

– Let $(pc_{\langle v_1,v_2\rangle}, \beta_1, \beta_2, f_1, f_2, p_1, p_2)$ be the corresponding entry in $\mathcal{E}_p$. If such entry does not exist, abort.
– Set $\mathbb{B}[v_1] = \mathbb{B}[v_1] + \beta_1$, $\mathbb{B}[v_2] = \mathbb{B}[v_2] + \beta_2$
– Remove from $\mathcal{E}_p$: $(pc_{\langle v_1,v_2\rangle}, \beta_1, \beta_2, f_1, f_2, p_1, p_2)$

$\underline{openVC\ (c_{\langle v_1,v_I\rangle}, c_{\langle v_I,v_2\rangle}, \beta_1, \beta_2, f_1, f_2, p_1, p_2, f_e)}$

$c_{\langle v_1,v_I\rangle}$, $c_{\langle v_I,v_2\rangle}$: PC or VC identifiers
$\beta_{i\in\{1,2\}}$: Initial VC balance
$f_{i\in\{1,2\}}$: Base routing fee
$p_{i\in\{1,2\}}$: Proportional r. fee
f_e: Establishing fee

– Let $(c_{\langle v_1,v_I\rangle}, \beta'_1, \beta'_I, f'_1, f'_I, p'_1, p'_I)$ and $(c_{\langle v_I,v_2\rangle}, \beta''_I, \beta''_2, f''_I, f''_2, p''_I, p''_2)$ be the entries in $\mathcal{E}_p$ or $\mathcal{E}_v$ corresponding to $c_{\langle v_1,v_I\rangle}$ and $c_{\langle v_I,v_2\rangle}$. If $(\beta_1 + f_e) > \beta'_1$ or $\beta_2 > \beta'_I$ or $\beta_2 > \beta''_2$ or $\beta_1 > \beta''_I$, abort.
– Update the entries in $\mathcal{E}_p$ or $\mathcal{E}_v$ as $(c_{\langle v_1,v_I\rangle}, \beta'_1 - (\beta_1 + f_e), \beta'_I - \beta_2 + f_e, f'_1, f'_I, p'_1, p'_I)$ and $(c_{\langle v_I,v_2\rangle}, \beta''_I - \beta_1, \beta''_2 - \beta_2, f''_I, f''_2, p''_I, p''_2)$
– Add $(vc_{\langle v_1,v_2\rangle}, \beta_1, \beta_2, f_1, f_2, p_1, p_2)$ in $\mathcal{E}_v$

$\underline{closeVC\ (vc_{\langle v_1,v_2\rangle})}$

$vc_{\langle v_1,v_2\rangle}$: VC identifier

– Let $(vc_{\langle v_1,v_2\rangle}, \beta_1, \beta_2, f_1, f_2, p_1, p_2)$ be the corresponding entry in $\mathcal{E}_v$. If such entry does not exist, abort.
– Update the entries in $\mathcal{E}_p$ as $(pc_{\langle v_1,v_I\rangle}, \beta'_1 + \beta_1, \beta'_I + \beta_2, f'_1, f'_I, p'_1, p'_I)$ and $(pc_{\langle v_I,v_2\rangle}, \beta''_I + \beta_1, \beta''_2 + \beta_2, f''_I, f''_2, p''_I, p''_2)$
– Remove $(vc_{\langle v_1,v_2\rangle}, \beta_1, \beta_2, f_1, f_2, p_1, p_2)$ in $\mathcal{E}_v$

$\underline{update\{P,V\}C\ (c_{\langle v_1,v_2\rangle}, \beta)}$

$c_{\langle v_1,v_2\rangle}$: Channel identifier

– Let $(c_{\langle v_1,v_2\rangle}, \beta_1, \beta_2, f_1, f_2, p_1, p_2)$ be the corresponding entry in $\mathcal{E}_p \cup \mathcal{E}_v$. If such entry does not exist or it exists but $\beta_1 < \beta$, abort.
– Update the payment or virtual channel as follows $(c_{\langle v_1,v_2\rangle}, \beta_1 - \beta, \beta_2 + \beta, f_1, f_2, p_1, p_2)$

$\underline{pay\ ((c_{\langle s,v_1\rangle}, \ldots, c_{\langle v_n,r\rangle}), \beta)}$

$(c_{\langle s,v_1\rangle}, \ldots, c_{\langle v_n,r\rangle})$: List of channels
β: Payment amount

– The channels must form a path from sender s to receiver r. Otherwise, abort.
– For each channel $(c_{\langle v_i,v_{i+1}\rangle}, \beta_i, \beta_{i+1}, f_i, f_{i+1}, p_i, p_{i+1})$, $i = 0, 1, \ldots, n$ ($s = v_0$ and $r = v_{n+1}$), it must be the case that $\beta_i \geq send_i$. Otherwise, abort.
– Update each channel in the path as follows $(c_{\langle v_i,v_{i+1}\rangle}, \beta_i - send_i, \beta_{i+1} + send_i, f_i, f_{i+1}, p_i, p_{i+1})$, $i = 0, 1, \ldots, n$.
Recursive definitions for a payment path $(s = v_0, v_1, \ldots, v_n, r = v_{n+1})$.
$send_\ell$, $\ell = 0, \ldots, n$, is the amount that node v_ℓ sends to $v_{\ell+1}$: $send_\ell = \beta + \sum_{i=\ell+1}^{n} fee_i$.
fee_ℓ, $\ell = 1, \ldots, n$, is the amount that node v_ℓ charges (keeps) for forwarding the payment $fee_\ell = f_\ell + p_\ell \cdot send_\ell = f_\ell + p_\ell(\beta + \sum_{i=\ell+1}^{n} fee_i)$. We assume that a sum that starts from a higher index than its ending index equals zero.

Fig. 2: Operations in a VPCN.

3 Modelling virtual payment channel networks

We introduce a more formal model of virtual payment channel networks (VPCNs). We will then discuss the security and privacy threats by on-path adversaries, define the studied optimization goal on VPCNs, and show its NP-hardness.

Definition 1 (VPCN). *A virtual payment channel network, VPCN, is defined as a graph $\mathcal{G} := (\mathcal{V}, \mathcal{E})$ where $\mathcal{V}$ denotes the set of users in the network*

and $\mathcal{E} := \mathcal{E}_p \cup \mathcal{E}_v$ denotes the set of channels. In particular, $\mathcal{E}_p$ denotes the set of payment channels and $\mathcal{E}_v$ denotes the set of virtual channels. Each payment channel is defined by a tuple $(pc_{\langle v_1,v_2\rangle}, \beta_1, \beta_2, f_1, f_2, p_1, p_2)$[2], where $pc_{\langle v_1,v_2\rangle}$ denotes a payment channel identifier, $\beta_{i\in\{1,2\}}$ denotes the current balance of the node $v_{i\in\{1,2\}}$, $f_{i\in\{1,2\}}$ is the base fee and $p_{i\in\{1,2\}}$ the fee rate (proportional to the amount paid) charged to use this channel in each direction, respectively. Analogously, a virtual channel is defined by a tuple $(vc_{\langle v_1,v_2\rangle}, \beta_1, \beta_2, f_1, f_2, p_1, p_2, f_e)$, where f_e denotes the VC establishment fee.

A VPCN is defined with respect to a blockchain $\mathbb{B}$ that stores publicly accessible entries of the form $(v, \beta^{on\text{-}chain})$ where v denotes an address of the underlying blockchain and $\beta^{on\text{-}chain}$ denotes its on-chain balance. For readability, we hereby use $\mathbb{B}[v]$ to denote the on-chain balance of v in $\mathbb{B}$.

A VPCN exposes the operations expressed in Figure 2.

Security and privacy for on-path adversaries. On-path adversaries may cause a diverse set of attacks in PCNs [11,22,23,27,37,40,41] and with significant impact [38]. We employ VCs to defend against on-path adversaries, by bypassing corrupted nodes (cf. Figure 1). We chose to investigate three representative attacks: value privacy, relationship anonymity, and wormhole attacks [22, 23]. We chose these attacks because they are well studied in the literature and note that our approach directly generalizes to other on-path adversarial attacks, such as denial-of-service attacks [40], which we later show in Section 4.3.

We assume that the adversary has a budget $\mathcal{B}$ for corrupting nodes and uses a deterministic function for selecting which nodes to attack, based on the budget and public information about the PCN. Honest nodes are not aware of $\mathcal{B}$ or the set of nodes $\mathcal{X}$ attacked by the adversary, but use public information about the PCN to estimate the adversarial strategy and $\mathcal{B}$, which in turn outputs a set of *potentially* corrupted nodes, say $\tilde{\mathcal{X}}$. Note that possibly $\mathcal{X} \neq \tilde{\mathcal{X}}$.

We compute $\tilde{\mathcal{X}}$ to be the optimal set of nodes for an adversary to corrupt, such that the largest fraction of payments is affected, based on previous works [18,20,28,38]. These works analyze the optimal placement of adversaries in order to maximize the affected channels by the attacks. Moreover, the budget can be estimated as a fraction of the total channel capacities. This static approach is for the honest nodes to deny an adversary exactly this optimal placement within the graph, by bypassing these nodes. By denying adversaries the nodes where they can do the most damage, the overall security and privacy against on-path attacks is improved, attacks become more expensive and finally, if we establish a direct VC between two end-users, we effectively prevent on-path attacks.

For a path $s - u_1 - \ldots - u_n - r$ the attacks on value privacy, on relationship anonymity, and the wormhole attack are defined as described as follows.

– *Value privacy* [22]: PCN payments ensure that the transaction amount remains private to off-path corrupted users if there are only honest users along the path. This means, that if there are on-path corrupted users, value privacy

[2] In [22], a payment channel also contains a timeout parameter. In practice is no longer required [30] and thus we omit it here.

does not hold anymore. *Preventing this attack:* For all segments $u_i - x_1 - x_2 - \ldots - x_\ell - u_j$ of the path from s to r, where u_i, u_j are not corrupted and x_p, $p = 1, \ldots, \ell$ are corrupted, build a virtual channel from u_i to u_j.

– *Relationship anonymity* [22]: If an adversary controls two corrupted users u_1 and u_n, they can distinguish who is paying to whom. *Preventing this attack:* If $u_i - x_1 - x_2 - \ldots - x_\ell - u_j$ is a segment of the path from s to r, where u_i, u_j are not corrupted, x_p, $p = 1, \ldots, \ell$ are corrupted and $u_i \in \{s, r\} \vee u_j \in \{s, r\}$. If both s and r are part of such a segment, take one segment (there can be at most two) and build a virtual channel from u_i to u_j.

– *Wormhole attack* [23]: In PCN payments, an adversary can prevent honest users from finalizing payments and effectively steal their fees. For this, the adversary needs to control corrupted nodes on both sides of one or more honest nodes along the path. *Preventing this attack:* Identify all segments $u_i - x_1 - \ldots - x_\ell - y_1 - \ldots - y_m - z_1 - \ldots - z_n - u_j$ of the path from s to r where u_i, u_j and y_p, $p = 1, \ldots, m$ are not corrupted and where x_q, $q = 1, \ldots, \ell$ and z_r, $r = 1, \ldots, n$ are corrupted. For each segment, build one of the following virtual channels: (i) between $u_i - y_1$ (ii) between $y_m - u_j$ (iii) between $x_\ell - z_1$.

Costs of VCs. Once opened, VCs can effectively reduce the fees of payments within a VPCN, as we explained in Section 2. However, to create a VC, the endpoints need to pay an establishment fee f_e. Since VCs are currently not used, there is no fee model in practice which we can use. We therefore assume that f_e of a VC over some path with capacity α to be the same as users would charge for forwarding a payment of amount α over that path. I.e., node v_j charges $fee_j = f_{j,j+1} + p_{j,j+1} \cdot (\alpha + \sum_{k=j+1}^{n} fee_k)$, for $j = 1, \ldots, n$. We discuss other potential fee models in Appendix F and note that f_e is modular in our model and can easily be defined alternatively.

Optimization goal. Our objective is to set up virtual channels such that the cost for routing a set of transactions is minimized, and no transaction is traversing a path prone to an attack (based on our estimation of the adversarial strategy and budget). The main optimization goal of our work and its hardness appear follow (the NP-hardness proof appears in Section B).

Definition 2 (VPCN cost optimization). *Given a VPCN $\mathcal{G}$, a set of transactions $\mathcal{T}$, an estimated strategy of an on-path adversary to corrupt nodes and the estimated budget of the adversary $\mathcal{B}$ for doing so, minimize the cost for routing the transactions in $\mathcal{T}$, such that no transaction is traversing a path that is prone to a given attack. If the estimation of the adversary's budget is $\mathcal{B} = 0$ our goal is to minimize the routing fees.*

Theorem 1. *The VPCN cost optimization problem is NP-hard.*

4 Algorithms and analysis

This section presents two optimization approaches; an integer linear program and an efficient greedy algorithm. We also discuss the synergy among objectives.

4.1 A rigorous algorithm

A rigorous and exact approach to optimize the virtual payment channel network (VPCN) is to use integer linear programming (ILP). The optimization goal (objective function) is to minimize the cost of routing all transactions, while eliminating all attacks of Section 3. The constraints ensure that transactions are executed via unique payment paths, capacities are respected, and virtual channels (VC) can be created over two other channels. These constraints define a set (polytope) of feasible solutions and the output of an ILP solver is the minimum value of the objective function within this set (if it is not empty).

Input. The input needed to define the ILP is a PCN (as defined in Section 2) including all the channels and their attributes, a set of T transactions $\mathcal{T} = \{transaction_t = (s_t, d_t, trans_t)\}_{t\in[1,T]}$, i.e. (source, destination, amount), the estimated budget $\mathcal{B}$ of the adversary, and a constant $c_{tr} \in [0, 1]$ indicating the required minimum fraction of total successful transaction amounts (e.g. if $c_{tr} = 1$ all transactions must be executed). Let $ch_{ij}.base_fee$ (f_i in Definition 1) and $ch_{ij}.prop_fee$ (p_i in Definition 1) denote the base and proportional forwarding fee of a payment ($ch_{ij} = pc_{ij}$) or a virtual channel ($ch_{ij} = vc_{ij}$), and $pc_{ij}.capacity$ denote the payment channel (PC) capacity (β_i in Definition 1).

We set the virtual channel fees to be equal to those of the underlying initial payment channel. Since a virtual channel vc^k_{ij} can be constructed over any combination of two adjacent payment or virtual channels ch_{ik} and ch_{kj}, we assume a recursive structure of VCs and bound the levels of recursion by the input parameter w. The set of nodes which we need to bypass with virtual channels is part of the input as well, as we compute it by fixing the estimated (by the honest nodes) set of corrupted nodes $\tilde{\mathcal{X}}$, the budget $\mathcal{B}$, and the attack type (Section 3).

Constants and variables. We indicate the success of $transaction_t$ with $trans_t_succ \in \{0, 1\}$. Let $\mathcal{P}(s_t, d_t)$ be a list of all the paths from a sender s_t to a receiver d_t, for $transaction_t$, over the existing payment and all possible virtual channels (of level at most $w - 1$). The variable $path_P(trans_t) \in \{0, 1\}$ indicates whether $transaction_t$ is routed through path $P \in \mathcal{P}(s_t, d_t)$. Let $routing_fee(t, P, ch_{ij})$, $ch_{ij} \in \{pc_{ij}, vc_{ij}\}$, be the routing fees of a channel $ch_{ij} \in P$ for $transaction_t$. We note that $routing_fee(t, P, ch_{ij})$ is computed as in Section 2 if $ch_{ij} \in P$ and is zero otherwise. Let $routing_cost_{ch_{ij}} = \sum_{t=1}^{T} \sum_{P\in\mathcal{P}(s_t,d_t)} routing_fee(t, P, ch_{ij}) \cdot path_P(trans_t)$ be the routing fees that are charged for the transactions that traverse channel $ch_{ij} \in \{pc_{ij}, vc_{ij}\}$.

We assume that a level 0 virtual channel is built over two payment channels and a level m virtual channel is built recursively over two payment or virtual channels of level at most $m - 1$. We define the variables for (recursive) virtual channels and their capacities as follows. Let $L_0_vc^k_{ij} \in \{0, 1\}$ indicate the existence of a level-0 virtual channel, i.e., using two payment channels, between i, j via k, and $L_0_vc^k_{ij}.capacity$ denote its capacity. Let $L_{mpv}_vc^k_{ij}, L_{mvp}_vc^k_{ij}$, $L_{mvv}_vc^k_{ij} \in \{0, 1\}$ indicate the existence of a virtual channel between i, j via k, built upon a level-$(m-1)$ virtual channel and either a payment or a level-r virtual channel, with $r \leq m-1$. The subscripts pv, vp, vv denote each of the three cases:

p denotes a payment channel, v a virtual channel, the first letter refers to the nature of the channel between i, k and the second to the one of k, j. For each of these cases, the capacities are denoted by $L_{mpv}_vc_{ij}^k.capacity$, $L_{mvp}_vc_{ij}^k.capacity$, and $L_{mvv}_vc_{ij}^k.capacity$. We define the estimated capacity of a virtual channel as follows: $vc_{ij}.capacity = \sum_{k \neq i,j}[L_0_vc_{ij}^k.capacity + \sum_{q=1}^{w-1}(L_{qpv}_vc_{ij}^k.capacity + L_{qvp}_vc_{ij}^k.capacity + L_{qvv}_vc_{ij}^k.capacity)]$ (only one node k will be used for creating this virtual channel). We define the virtual channel creation cost as follows: $vc_{ij}_creation_cost = vc_{ij}.base_fee + vc_{ij}.prop_fee \cdot vc_{ij}.capacity$.

Objective. The objective is to minimize routing and virtual channel creation costs: $\min \sum_{i,j} routing_cost_{pc_{ij}} + \sum_{i,j}(routing_cost_{vc_{ij}} + vc_{ij}_creation_cost)$.

Constraints: capacities, success ratio, payment paths.
(C1) Payment and virtual channel capacities should be respected:
$routing_cost_{vc_{ij}} + trans_amount(vc_{ij}) \leq vc_{ij}.capacity$ and
$vc_{ij}_creation_cost + routing_cost_{pc_{ij}} + trans_amount(pc_{ij}) \leq pc_{ij}.capacity$,
where $trans_amount(ch_{ij}) = \sum_{t=1}^{T} \sum_{P \in \mathcal{P}(s_t,d_t)} trans_t \cdot path_P(trans_t) \cdot In(ch_{ij}, P)$ and $In(ch_{ij}, P)$ is a constant indicating whether $ch_{ij} \in P$.
(C2) The percentage of successful transaction amounts should be at least c_{tr}: $\sum_{t=1}^{T} trans_t \cdot trans_t_succ \geq c_{tr} \sum_{t=1}^{T} trans_t$.
(C3) A transaction is successful iff there is exactly one path routing it:
$\sum_{P \in \mathcal{P}(s_t,d_t)} path_P(trans_t) = trans_t_succ$, $\forall t \in [1, T]$.
If possible by the capacities (C1), C2 forces the ILP solver to set some $trans_t_succ$ variables to 1, which by C3 forces some paths to be selected for routing transactions. C2 also prohibits the solver to return a solution where no transaction is successful, since this trivial solution returns the minimum routing fees.

Constraints: virtual channel existence and capacities. Constraint C4 includes the constraints for level-0 VCs. The constraints for level-m VCs, $m \in [1, w-1]$, are a direct generalization of the ones for level-0 VCs and are presented in Appendix C.1 (constraints C5–C7). The key to this generalization is distinguishing the three possible combinations of underlying channels for building a level-m VC: (PC,VC) (C5), (VC,PC) (C6), or (VC,VC) (C7), where each VC used has at most $m-1$ recursion levels and one has exactly that many. C8 requires that at most one VC of any level exists between each pair of nodes.
(C4) Level-0 VC:
(i) A level-0 vc_{ij}^k can only be created over two PCs: $2 \cdot L_0_vc_{ij}^k \leq pc_{ik} + pc_{kj}$.
(ii) The VC capacity is bounded by the capacity of the used payment channels: $L_0_vc_{ij}^k.capacity \leq pc_{ik}.capacity$ and $L_0_vc_{ij}^k.capacity \leq pc_{kj}.capacity$
(iii) No capacity if $L_0_vc_{ij}^k = 0$: $L_0_vc_{ij}^k.capacity \leq L_0_vc_{ij}^k \cdot \max pc_{ij}.capacity$
(iv) At most one VC from i to j can be created: $\sum_{k \in V} L_0_vc_{ij}^k \leq 1$.
(C9) $exists_vc_{ij}^k := L_0_vc_{ij}^k + \sum_{\ell=1}^{w-1} \left(L_{\ell pv}_vc_{ij}^k + L_{\ell vp}_vc_{ij}^k + L_{\ell vv}_vc_{ij}^k\right) \leq 1$.
(C10) A virtual channel can be used only if it exists $vc_{ij}^k(trans_t) \leq exist_vc_{ij}^k$.

Constraints: eliminating attacks. Recall that the set of virtual channels needed for bypassing the estimated corrupted nodes to eliminate a given attack is part of the input. For each such VC, we set the constraint $exists_vc_{ij}^k = 1$.

Challenges. Designing an ILP to solve VPCN cost optimization exactly is thus feasible, however this approach is impractical. The ILP has exponentially many variables and constraints (asymptotically), since it needs to decide which subset of all paths minimizes the objective. However, an efficient variant (e.g. one that restricts the number of possible paths to be polynomially many) or implementation of the ILP can be used as a globally optimal solution. The drawback of including the list of transactions in the input can be alleviated using techniques for computing the ILP in a privacy-preserving manner, as in [39].

4.2 A fast algorithm

In the following, we hence present an efficient greedy algorithm approximating the optimal solution which has a low running time while ensuring high-quality channel allocations (see also the upcoming empirical evaluation).

Our greedy algorithm entirely bypasses some or all on-path adversaries. While we will present our strategies in more detail in Algorithms 4 to 6 in Appendix E, we give a short outline in the following. For relationship anonymity, it is sufficient to greedily bypass corrupted nodes adjacent to either the sender or the receiver, along short paths. To prevent the wormhole attack, corrupted nodes need to be bypassed such that there are no honest nodes encased by corrupted nodes. For value privacy, all corrupted nodes need to be bypassed.

Furthermore, regarding fees optimization, we recall that we model the fee for opening a VC of capacity α is the same as routing a payment of amount α via that path. This means that in our model it is cheaper to construct a direct VC between sender and receiver, as soon as we carry out a payment on a path more than once. We show this strategy in more detail in Algorithm 7 in Appendix E.

4.3 Synergy among objectives

From the definitions of the three different attacks and the strategies of how to prevent them, it becomes apparent that some strategies entail others. More concretely, preventing value privacy attacks also prevents attacks on relationship anonymity and wormhole attacks. Furthermore, following our fee optimization heuristic (see Section 5) also prevents all three security and privacy attacks.

On a high level, for a given path p, let $V(p), R(p), W(p), F(p)$ be one out of potentially multiple sets of nodes that are at least required to be bypassed for preventing value privacy attacks, relationship anonymity attacks, and wormhole attacks, as well as for optimizing the fees, respectively. That is, if these nodes or a superset of them are bypassed, the corresponding attack is prevented. Formally, we define these sets as the return values of Algorithms 4 to 7 in Appendix E.

Theorem 2. *For any path p, $F(p) \supseteq V(p) \supseteq R(p)$ and $F(p) \supseteq V(p) \supseteq W(p)$.*

We prove this theorem in Section C.3. Note that both by preventing value privacy attacks and by optimizing for fees, we bypass every corrupted node on the path. It follows that from these two strategies we also get security against any type attack that relies on having at least one corrupted node on the path, such as for instance denial of service attacks on payments.

5 Empirical evaluation

We conducted extensive simulations to shed light on the optimized setup and structure of virtual channels, as well as to study the performance of our algorithm. In order to study networks at scale, we use on our efficient algorithm as our tool in the following.

5.1 Input data preparation and methodology

Graph model and data. Recalling our model in Section 3, let $\mathcal{G} := (\mathcal{V}, \mathcal{E} := \mathcal{E}_p \cup \mathcal{E}_v)$ be our VPCN graph with $\mathcal{V}$ the set of nodes, $\mathcal{E}_p$ the set of payment channels and $\mathcal{E}_v$ (initially $\emptyset$) the set of virtual channels.

We conduct our experiments on a snapshot of the Lightning Network (LN) from March 4, 2021, extracted from `https://ln.fiatjaf.com/`. The graph contains 33k channels and 8k nodes that are part of at least one channel. For each channel, we read the capacity, the base and the relative fee. The total network capacity is $1,167.4$ BTC, the average base fee is $3,165$ msat (millisatoshi), the average relative fee rate is $32,417$ millionth of the satoshis transferred (one BTC is 100M satoshi). Due to the nature of PCNs the individual balance of each user remains private to an outsider. This is a common limitation for works investigating PCNs. We assume that each channel capacity is initially evenly distributed between both nodes.

Payments. We generate payments selecting sender-receiver pairs by sampling two random nodes in the graph that are connected via a path. For each sender-receiver pair the shortest path in terms of fees is chosen using Dijkstra's algorithm with a custom weight function that computes the fee charged as a sum of the base fee and proportional fee times the amount. We sample $r_pay = 100$ payments along with a random payment amount $val \in [1, 10]$ satoshis, modelling a micro-payment setting since the average channel capacity is $2.6M$ satoshis.

Constructing virtual channels. We create VCs on top of the PCN, both for direct payments between endpoints and for routing other payments through the VCs. As VCs are currently not used in PCNs such as the Lightning Network, there exist no fee model in practice. In the context of VCs, it seems that users can charge fees for (i) establishing the VC if they are intermediaries or (ii) for routing payments through the VC if they are endpoints.

To model the VC establishment fees, we interpret the base fee as what a hop charges for actively participating in the protocol, and the fee rate as what a hop charges for locking up α coins, i.e., the opportunity cost of that node. We model the establishment fee of a VC with capacity α to be the same as routing a payment of α coins via that path (see Section 3). We discuss other possible fee models in Appendix F. For the VC routing fee, we take the one of the underlying payment channel of the VC, in which the initiating endpoint is involved.

Corruption model. Honest nodes assume there is an attacker who has an estimated budget and who corrupts nodes according to an estimated strategy. In order to corrupt a node, we say an attacker needs to spend the money that

this node has locked up in its neighboring channels. We define this locked up money for any $v \in \mathcal{V}$ as $capacity_locked(v) := \sum_{w \in \mathcal{V} \setminus \{v\}} (pc_{\langle v,w \rangle} . \beta_1)$.

The assumptions that an attacker corrupts nodes that are most beneficial to it, while being cheap to place, is based on previous works [18, 20, 28, 38]. We parameterize the estimated adversary budget adv_budget as a percentage of the total capacity in all edges of $\mathcal{G}$. The absolute adversary capacity is $adv_capacity := adv_budget \cdot total_network_capacity$. To choose the best placed nodes, the adversary computes random payment paths and selects those nodes that appear most often on these paths. Note that no payment is actually carried out, only the paths are computed to find the most used nodes. Let P be a list of num_pay (e.g., 500) randomly chosen payment paths (i.e., path of connected edges) in $\mathcal{G}$. For every node $v \in \mathcal{V}$, we let $occ(v)$ be the number of their occurrence in P. We define the following cost-benefit ratio for every node v as follows: $cost_benefit(v) := \frac{occ(v)/num_pay}{capacity_locked(v)/adv_capacity}$.

Let l be a list of every node $v \in \mathcal{V}$ sorted by their cost-benefit ratio in descending order. We determine the list of all corrupted nodes C iterating over all the nodes and adding those for which the following condition holds after adding them: $\sum_{n \in C} (capacity_locked(n)) \leq adv_budget$

Observation: Repeating payments needed for VCs to be cost efficient. If a VC is used only once, it will never cost fewer fees than routing a payment directly through the underlying PCs. Therefore, we investigate the effect of conducting our r_pay payments multiple times.

Measuring the cost benefit in terms of fees. The cost of routing the r_pay payments through the payment channel network without virtual channels is denoted as $route_pcn$. The cost of establishing the virtual channels to prevent a certain type of attack is denoted as $establish_vc$. The routing cost when using the virtual channels is $route_vc$. We are interested in how the following ratio progresses as we increase the number of times that payments are repeated: $fee_ratio := \frac{establish_vc + route_vc}{route_pcn}$.

Measuring the effects of VCs on security and privacy. We measure how many payment paths are prone to a certain attack, with and without the VCs.

5.2 Results

We first study the benefit of optimizing VCs for either of our goals, and the implications on other goals. To shed light on this matter, we fix an adversary budget and corrupt the nodes according to our corruption model. For each payment we then use VCs (i) to optimize for security or privacy by preventing one of these attacks completely if that payment path is prone to that attack or (ii) to optimize for fees, both according to the algorithms presented in Section 4.2. Finally, we measure the impact this has on both the other security and privacy attacks as well as the fees. The algorithms used here can be found in Appendix E.

In our experiment, we investigate value privacy, relationship anonymity and wormhole attacks. For these experiments we need to choose an adversary budget that results in meaningful security threats from all these attacks. By meaningful

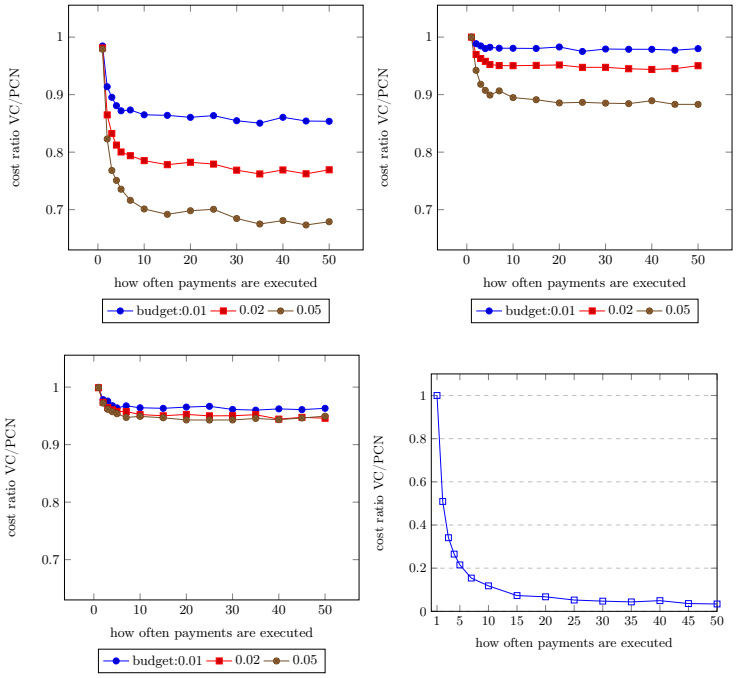

Fig. 3: Optimizing for value privacy (top left), relationship anonymity (top right), wormhole attack (bottom left) and fees (bottom right)

we mean that some of our paths (not 0 and not all of them) are susceptible to each of the three different attacks. For this, we need to compute the percentage of paths that are prone to which attack for different adversary budgets. We explain this in more detail in Appendix D and end up choosing 1, 2 and 5%.

Q1: How does preventing one attack affect the money spent on fees? To answer this question, we first measure the cost of routing payments through the PCN without VCs as a baseline. Then, we construct virtual channels with different goals in mind: bypass all corrupted nodes for value privacy, relationship anonymity and the wormhole attack. After constructing the VCs, we measure the cost again. We measure the ratio according to our definition in Section 5.1. We conduct each experiment 100 times and compute the average, with the results shown in Figure 3. We observe, that for all three budgets the cost ratio starts out around 1 for one payment. As the number of repeating payments goes up, the cost ratio decreases because the VCs are more effective. Moreover, the more nodes are corrupted, the better this ratio becomes when there are more payments. Note that the different effect on the ratio has to do mostly with the number of affected payments due to corrupted nodes and subsequently the number of VCs constructed. For relationship anonymity, this ratio goes down to 0.88, wormhole attack to 0.95, for value privacy to 0.68.

We can see from these results that, generally speaking, bypassing nodes to prevent each attack has a positive effect on the fee ratio, if the payments are repeated more than once. The best effect can be seen from preventing value privacy attacks. Also, the ratio goes down more steeply for the first 10 payments, afterwards the effect is more flat.

Q2: How does optimizing for fees affect the money spent on fees? Similar to when optimizing for security and privacy goals, we observe a steep decline on the ratio of money spent for fees when constructing VCs to the money spent for fees if we do not construct VCs. This ratio halves if there are 2 sequential payments and continues to drop to 0.04 for 50.

Q3: How do the different optimization strategies affect the security and privacy? We already compared the effect of the strategies optimizing the different goals on the fees. Now we want to evaluate the effect that they have on the security and privacy goals. For this, we measure how many of our payment paths are prone to each of the different attacks. Then we construct the VCs optimizing each goal and measure how many paths are prone then. In Figure 4 we show for each optimization strategy, (i) how many VCs are constructed, (ii) the average length of each VC and (iii) for each attack type two values $x; y$, where x is the percentage of paths prone to the attack before building VCs and y is the percentage of paths prone to the attack after building VCs. We notice that optimizing for value privacy also prevents the attacks on relationship anonymity and the wormhole attack. Furthermore, optimizing for fees prevents all three attacks we investigate. These results are in line with Section 4.3.

Fig. 4: How many paths are prone to different attacks when optimizing for different goals for an adversary budget of 0.05.

			Paths prone to (PCN; VC)		
Optimizing	# VC	Avg VC length	VP attacks	RA atk.	WH atk.
VP	126	3.1	97; 0	21; 0	35; 0
RA	21	3.6	97; 84	21; 0	35; 28
WH	33	2.2	97; 97	21; 13	35; 0
Fees	100	5.6	97; 0	21; 0	35; 0

6 Conclusion

Motivated by the potential benefits of virtual channels, to reduce transaction fee cost as well as to improve security and privacy guarantees in payment channel networks, we presented a first systematic study of the virtual channel setup problem. We have shown that the problem can be formulated as an optimization problem, and described an integer linear program (ILP) accordingly. While the ILP can give exact solutions, we proved that the problem is NP-hard in general, and presented a fast greedy algorithm. Using simulations on the Lightning Network, we provided insights into the structure of virtual channels and confirmed the benefits of our optimization approach.

We believe that our work opens several interesting avenues for future research. In particular, it would be interesting to study different fee models or the effect of timing, i.e., adding time frames in which transactions are to be executed as well as VC lifetimes.

References

1. Aumayr, L., Maffei, M., Ersoy, O., Erwig, A., Faust, S., Riahi, S., Hostáková, K., and Moreno-Sanchez, P. Bitcoin-compatible virtual channels. In *2021 IEEE Symposium on Security and Privacy (SP)* (May 2021), IEEE Computer Society, pp. 901–918.
2. Aumayr, L., Moreno-Sanchez, P., Kate, A., and Maffei, M. Blitz: Secure multi-hop payments without two-phase commits. In *30th USENIX Security Symposium (USENIX Security 21)* (Aug. 2021), USENIX Association, pp. 4043–4060.
3. Aumayr, L., Moreno-Sanchez, P., Kate, A., and Maffei, M. Donner: Utxo-based virtual channels across multiple hops. Cryptology ePrint Archive, Report 2021/855, 2021. `https://ia.cr/2021/855`.
4. Avarikioti, G., Kogias, E. K., and Wattenhofer, R. Brick: Asynchronous state channels. *arXiv preprint arXiv:1905.11360* (2019).
5. Avarikioti, G., Scheuner, R., and Wattenhofer, R. Payment networks as creation games. In *Data Privacy Management, Cryptocurrencies and Blockchain Technology - ESORICS 2019 International Workshops, DPM 2019 and CBT 2019, Luxembourg, September 26-27, 2019, Proceedings* (2019), C. Pérez-Solà, G. Navarro-Arribas, A. Biryukov, and J. García-Alfaro, Eds., vol. 11737 of *Lecture Notes in Computer Science*, Springer, pp. 195–210.
6. Avarikioti, Z., Heimbach, L., Wang, Y., and Wattenhofer, R. Ride the lightning: The game theory of payment channels. In *Financial Cryptography and Data Security - 24th International Conference, FC 2020, Kota Kinabalu, Malaysia, February 10-14, 2020 Revised Selected Papers* (2020), J. Bonneau and N. Heninger, Eds., vol. 12059 of *Lecture Notes in Computer Science*, Springer, pp. 264–283.
7. Dotan, M., Pignolet, Y.-A., Schmid, S., Tochner, S., and Zohar., A. Survey on blockchain networking: Context, state-of-the-art, challenges. In *Proc. ACM Computing Surveys (CSUR)* (2021).
8. Dziembowski, S., Eckey, L., Faust, S., and Malinowski, D. Perun: Virtual Payment Hubs over Cryptocurrencies. In *2019 IEEE Symposium on Security and Privacy (SP)* (May 2019), pp. 106–123.
9. Egger, C., Moreno-Sanchez, P., and Maffei, M. Atomic Multi-Channel Updates with Constant Collateral in Bitcoin-Compatible Payment-Channel Networks. Tech. Rep. 583, 2019.
10. Eilam-Tzoreff, T. The disjoint shortest paths problem. *Discrete applied mathematics 85*, 2 (1998), 113–138.
11. EmelyanenkoK. Payment channel congestion via spam-attack. In *https://github.com/lightningnetwork/lightning-rfc/issues/182* (2020).
12. Ersoy, O., Roos, S., and Erkin, Z. How to profit from payments channels. In *Financial Cryptography and Data Security - 24th International Conference, FC 2020, Kota Kinabalu, Malaysia, February 10-14, 2020 Revised Selected Papers* (2020), J. Bonneau and N. Heninger, Eds., vol. 12059 of *Lecture Notes in Computer Science*, Springer, pp. 284–303.
13. Green, M., and Miers, I. Bolt: Anonymous Payment Channels for Decentralized Currencies. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security* (New York, NY, USA, Oct. 2017), CCS '17, Association for Computing Machinery, pp. 473–489.
14. Gudgeon, L., Moreno-Sanchez, P., Roos, S., McCorry, P., and Gervais, A. Sok: Layer-two blockchain protocols. In *International Conference on Financial Cryptography and Data Security* (2020), Springer, pp. 201–226.

15. Heilman, E., Alshenibr, L., Baldimtsi, F., Scafuro, A., and Goldberg, S. Tumblebit: An untrusted bitcoin-compatible anonymous payment hub. In *Network and Distributed System Security Symposium* (2017).
16. Jourenko, M., Larangeira, M., and Tanaka, K. Lightweight virtual payment channels. In *Cryptology and Network Security - 19th International Conference, CANS 2020, Vienna, Austria, December 14-16, 2020, Proceedings* (2020), S. Krenn, H. Shulman, and S. Vaudenay, Eds., vol. 12579 of *Lecture Notes in Computer Science*, Springer, pp. 365–384.
17. Jourenko, M., Larangeira, M., and Tanaka, K. Payment Trees: Low Collateral Payments for Payment Channel Networks. Tech. Rep. 1313, 2020.
18. Kappos, G., Yousaf, H., Piotrowska, A., Kanjalkar, S., Delgado-Segura, S., Miller, A., and Meiklejohn, S. An Empirical Analysis of Privacy in the Lightning Network. In *Financial Cryptography and Data Security* (Jan. 2021).
19. Khamis, J., and Rottenstreich, O. Demand-aware channel topologies for off-chain payments. In *2021 International Conference on COMmunication Systems & NETworkS (COMSNETS)* (2021), IEEE, pp. 272–280.
20. Kumble, S. P., Epema, D., and Roos, S. How lightning's routing diminishes its anonymity. In *The 16th International Conference on Availability, Reliability and Security* (New York, NY, USA, 2021), ARES 2021, Association for Computing Machinery.
21. Malavolta, G., Moreno-Sanchez, P., Kate, A., and Maffei, M. Silentwhispers: Enforcing security and privacy in credit networks. In *Network and Distributed System Security Symposium* (2017).
22. Malavolta, G., Moreno-Sanchez, P., Kate, A., Maffei, M., and Ravi, S. Concurrency and privacy with payment-channel networks. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security* (2017), pp. 455–471.
23. Malavolta, G., Moreno-Sanchez, P., Schneidewind, C., Kate, A., and Maffei, M. Anonymous Multi-Hop Locks for Blockchain Scalability and Interoperability. In *26th Annual Network and Distributed System Security Symposium, NDSS 2019, San Diego, California, USA, February 24-27, 2019* (2019), The Internet Society.
24. McCorry, P., Bakshi, S., Bentov, I., Meiklejohn, S., and Miller, A. Pisa: Arbitration outsourcing for state channels. In *Proceedings of the 1st ACM Conference on Advances in Financial Technologies* (2019), pp. 16–30.
25. Moreno-Sanchez, P., Kate, A., Maffei, M., and Pecina, K. Privacy Preserving Payments in Credit Networks: Enabling trust with privacy in online marketplaces. In *22nd Annual Network and Distributed System Security Symposium, NDSS 2015, San Diego, California, USA, February 8-11, 2015* (2015), The Internet Society.
26. Neudecker, T., and Hartenstein, H. Network layer aspects of permissionless blockchains. *IEEE Communications Surveys & Tutorials 21*, 1 (2018), 838–857.
27. Nisslmueller, U., Foerster, K.-T., Schmid, S., and Decker, C. Toward active and passive confidentiality attacks on cryptocurrency off-chain networks. In *Proc. 6th International Conference on Information Systems Security and Privacy (ICISSP)* (2020).
28. Pickhardt, R., Tikhomirov, S., Biryukov, A., and Nowostawski, M. Security and privacy of lightning network payments with uncertain channel balances. *CoRR abs/2103.08576* (2021).

29. Pietrzak, K., Salem, I., Schmid, S., and Yeo, M. Lightpir: Privacy-preserving route discovery for payment channel networks. In *Proc. IFIP Networking* (2021).
30. Poon, J., and Dryja, T. The Bitcoin Lightning Network:, 2016.
31. Rohrer, E., Malliaris, J., and Tschorsch, F. Discharged payment channels: Quantifying the lightning network's resilience to topology-based attacks. In *2019 IEEE European Symposium on Security and Privacy Workshops (EuroS&PW)* (2019), IEEE, pp. 347–356.
32. Rohrer, E., and Tschorsch, F. Counting down thunder: Timing attacks on privacy in payment channel networks. In *AFT '20: 2nd ACM Conference on Advances in Financial Technologies, New York, NY, USA, October 21-23, 2020* (2020), ACM, pp. 214–227.
33. Romiti, M., Victor, F., Moreno-Sanchez, P., Nordholt, P. S., Haslhofer, B., and Maffei, M. Cross-Layer Deanonymization Methods in the Lightning Protocol. *arXiv:2007.00764 [cs]* (Feb. 2021).
34. Roos, S., Moreno-Sanchez, P., Kate, A., and Goldberg, I. Settling payments fast and private: Efficient decentralized routing for path-based transactions. In *25th Annual Network and Distributed System Security Symposium, NDSS 2018, San Diego, California, USA, February 18-21, 2018* (2018), The Internet Society.
35. Sivaraman, V., Venkatakrishnan, S. B., Alizadeh, M., Fanti, G. C., and Viswanath, P. Routing cryptocurrency with the spider network. In *Proceedings of the 17th ACM Workshop on Hot Topics in Networks, HotNets 2018, Redmond, WA, USA, November 15-16, 2018* (2018), ACM, pp. 29–35.
36. Tairi, E., Moreno-Sanchez, P., and Maffei, M. A\(\^{\}mbox2\)L: Anonymous Atomic Locks for Scalability and Interoperability in Payment Channel Hubs. *IACR Cryptol. ePrint Arch. 2019* (2019), 589.
37. Tang, W., Wang, W., Fanti, G., and Oh, S. Privacy-utility tradeoffs in routing cryptocurrency over payment channel networks. *Proceedings of the ACM on Measurement and Analysis of Computing Systems 4*, 2 (2020), 1–39.
38. Tikhomirov, S., Moreno-Sanchez, P., and Maffei, M. A quantitative analysis of security, anonymity and scalability for the lightning network. In *IEEE European Symposium on Security and Privacy Workshops, EuroS&P Workshops 2020, Genoa, Italy, September 7-11, 2020* (2020), IEEE, pp. 387–396.
39. Tiwari, S., Yeo, M., Avarikioti, Z., Salem, I., Pietrzak, K., and Schmid, S. Wiser: Increasing throughput in payment channel networks with transaction aggregation. *arXiv preprint arXiv:2205.11597* (2022).
40. Tochner, S., Zohar, A., and Schmid, S. Route hijacking and dos in off-chain networks. In *Proc. ACM Conference on Advances in Financial Technologies (AFT)* (2020).
41. Tripathy, S., and Mohanty, S. K. Mappcn: Multi-hop anonymous and privacy-preserving payment channel network. In *International Conference on Financial Cryptography and Data Security* (2020), Springer, pp. 481–495.
42. Zabka, P., Foerster, K.-T., Schmid, S., and Decker, C. A centrality analysis of the lightning network, 2022.

A Related work

Over the last years, significant research efforts have been devoted to the design and analysis of efficient and secure payment channel networks [7, 14, 26]. Motivated by topology-based attacks [31], the possibility of route hijacking [40] as well as vulnerabilities, e.g., related to the privacy [18, 27, 37] and anonymity of PCN users [32, 33], to just name a few examples, much existing literature revolves around network connectivity [31], the payment routing system [22, 40], as well as privacy aspects, e.g., of route discovery [29, 37].

To ensure anonymity, payment-channel networks usually rely on privacy enhancing cryptographic schemes (e.g., onion routing) to implement the 2-phase commit payment operation. PrivPay [25], SilentWhispers [21], Fulgor/Rayo [22], AHML [23] provide privacy-preserving multi-hop payment protocols which come with formal guarantees. SpeedyMurmurs [34] formalizes and addresses concrete notions of privacy in the context of payment routing. SpiderNetwork [35] improves the effectiveness of source routing in a dynamic PCN by favoring routes that minimize the balance difference using on-chain rebalancing. A privacy-preserving approach to discover low-cost routes was recently presented by Pietrzak et al. [29]. Blitz [2] is a 1-phase payment scheme, which similar to AMHL provides security against wormhole attacks [23]. None of the payment-based approaches, however, hide the value of the payment to intermediaries or decrease routing fees.

Therefore, an intriguing approach to improve the security and efficiency of payment channel networks is the use of virtual channels. These been introduced by Dziembowski et al. [8] to overcome the requirement that intermediaries along a channel route need to be online (a concern also considered in [4,24]) and explicitly confirm all mediated transactions. Recent work has extended the deployment scope of virtual channels, introducing efficient protocols that are compatible with Bitcoin and other popular cryptocurrencies [1, 3, 16].

While existing literature on virtual channels revolved around protocol design aspects, to the best of our knowledge, our paper is the first to investigate the problem of optimizing the allocation of virtual channels in order to improve the security and efficiency of PCNs. In parallel work [19], Khamis and Rottenstreich studied how to amortize the creation of new channels through reduced routing costs, however, without accounting for security aspects.

B NP-hardness of VPCN cost optimization

Unfortunately, the running time of the rigorous algorithm presented above is superpolynomial, and solving the integer program is computationally intractable in general.

Theorem 3 (Theorem 1, Section 3). *The VPCN cost optimization problem is NP-hard.*

Proof. We reduce an instance of the (NP-complete) minimum-length disjoint paths (MLDP) problem [10] to an instance of our VPCN problem. Consider an instance of the MLDP problem, i.e., an arbitrary directed graph $G = (V, E)$ such that $weight_G(u, v) = 1$, for all $(u, v) \in E$, and two[3] source destination pairs (s_1, d_1) and (s_2, d_2).

We now build an instance of the VPCN problem. We define $G' = (V, E \cup E')$, where $E' = \{(x, y) \mid (y, x) \in E \wedge (x, y) \notin E\}$, such that $weight_{G'}(x, y) = 1 + \varepsilon$, $\varepsilon = 1/|E|$, if $(x, y) \in E$ and $weight_{G'}(x, y) = 0$ if $(x, y) \in E'$. Thus, for every pair of nodes x, y in G' either $\{(x, y), (y, x)\} \in G'$ or x and y are not adjacent. The payment channels are hence defined as all the pairs of nodes x, y in G' such that $\{(x, y), (y, x)\} \in G'$, with capacity $weight_{G'}(x, y) + weight_{G'}(y, x)$, base fee equal to ε and proportional fee equal to 0. We consider the set of transactions, in the form of (source, destination, amount), to be $\{(s_1, d_1, 1), (s_2, d_2, 1)\}$. We also, assume that creating virtual channels is not possible, as the problem only becomes harder by including them. We set c_{tr}, the target percentage of successful transactions, to 1 (all should be executed).

A solution to the VPCN problem gives the minimum cost payment path for the two transactions. This set of payment paths in G' is using only edges that appear in G, as we set the capacity of the extra edges to zero and since the edge weights can accommodate for only one payment path, it does so with minimum length in G and also the paths are edge disjoint (the capacity suffices for only one transaction). Therefore a solution of the VPCN problem (payment paths) is exactly a solution of the minimum-length disjoint paths problem.

C Algorithms and Analysis: extra material

In this section we provide additional insights for our algorithms and analysis (Section 4). We present the full version of the ILP constratins about the virtual channel construction (Section C.1), an example solution for the ILP and the greedy algorithm (Section C.2), and the proof regarding the synergy among objectives (Section C.3).

C.1 Constraints for recursive virtual channels

We start by presenting the constraints related to constructing recursive virtual channels and assigning their capacities at level 0 (L_0), where two payment channels are used (PC, PC). As an intermediate (but not necessary) step, we present level 1 channels (L_1), which can be constructed by the combinations (PC, VC), (VC, PC), and (VC, VC), where a VC here is of level 0. Then we generalize from level 1 to level $m \in [1, w-1]$ (L_m), in which we have the same three options (PC, VC), (VC, PC), and (VC, VC), but each VC is of level at most $m - 1$. Finally, we present constraints and variables C9–C10, which relate to all recursion levels. A solution in which a virtual channel is constructed is preferable if it reduces

[3] Two pairs are enough to render the problem NP-complete [10].

the value of the objective function. Intuitively, this should happen when many transactions are routed through a payment path.

(C4) Level 0: (PC, PC). $L_0_vc_{ij}^k \in \{0,1\}$: a level 0 virtual channel from i to j exists using 2 payment channels from i to k and from k to j (pc_{ik}, pc_{kj}):

(i) vc_{ij}^k can only be created if there are 2 PC connecting i and j:

$$2 \cdot L_0_vc_{ij}^k \leq pc_{ik} + pc_{kj}$$

(ii) The capacity of this VC is bounded by the capacity of the used PC:

$$L_0_vc_{ij}^k.capacity \leq pc_{ik}.capacity$$
$$L_0_vc_{ij}^k.capacity \leq pc_{kj}.capacity$$

(iii) The capacity of a VC should be 0 if the existence variable is 0, i.e., $L_0_vc_{ij}^k = 0 \Rightarrow L_0_vc_{ij}^k.capacity = 0$. For this constraint a constant $max_cap = \max pc_{ij}.capacity$ (maximal channel capacity) is needed:

$$L_0_vc_{ij}^k.capacity \leq L_0_vc_{ij}^k \cdot max_cap$$

(iv) At most one VC from i to j can be created in every level

$$\textstyle\sum_{k \in V} L_0_vc_{ij}^k \leq 1$$

Note that this constraint and (iii) imply $\sum_{k \in V} L_0_vc_{ij}^k.capacity \leq max_cap$.

Level 1: (PC, VC). $L_{1pv}_vc_{ij}^k \in \{0,1\}$: a level 1 VC from i to j is constructed over pc_{ik} and an existing L_0 VC from k to j:

(i) vc_{ij}^k can only be created if there is a PC and a L_0 VC connecting i and j:

$$2 \cdot L_{1pv}_vc_{ij}^k \leq pc_{ik} + \textstyle\sum_{l \in V} L_0_vc_{kj}^l$$

(because of C8-iv, $\sum_{l \in V} L_0_vc_{kj}^l \leq 1$).

(ii) The capacity of a VC is bounded by the capacity of the used PC and VC

$$L_{1pv}_vc_{ij}^k.capacity \leq pc_{ik}.capacity$$
$$L_{1pv}_vc_{ij}^k.capacity \leq \sum_{l \in V} L_0_vc_{kj}^l.capacity$$

(second inequality on right-hand side: because of C8-iii and C8-iv at most one summand is not equal to 0).

(iii) $L_{1pv}_vc_{ij}^k = 0 \Rightarrow L_{1pv}_vc_{ij}^k.capacity = 0$ (if the existence variable is 0, then the corresponding capacity should also be 0), i.e.,

$$L_{1pv}_vc_{ij}^k.capacity \leq L_{1pv}_vc_{ij}^k \cdot max_cap$$

Level 1: (VC, PC). Symmetric to previous case:

$L_{1vp}_vc_{ij}^k \in \{0,1\}$: a level 1 VC from i to j exists, first using an existing L_0 virtual channel from i to k and then pc_{kj}

(i) vc_{ij}^k can only be created if there is a L_0 VC and a PC connecting i and j:

$$2 \cdot L_{1vp_}\, vc_{ij}^k \leq \sum_{l \in V} L_{0_}\, vc_{ik}^l + pc_{kj}$$

(ii) The capacity of a VC is bounded by capacity of the used VC and PC:

$$L_{1vp_}\, vc_{ij}^k.capacity \leq \sum_{l \in V} L_{0_}\, vc_{ik}^l.capacity$$
$$L_{1vp_}\, vc_{ij}^k.capacity \leq pc_{kj}.capacity$$

(Because of (ii) at most one summand on right-hand side in first inequality is not equal to 0).

(iii) $L_{1vp_}\, vc_{ij}^k = 0 \Rightarrow L_{1vp_}\, vc_{ij}^k.capacity = 0$

$$L_{1vp_}\, vc_{ij}^k.capacity \leq L_{1vp_}\, vc_{ij}^k \cdot max_cap$$

Level 1: (VC, VC). $L_{1vv_}\, vc_{ij}^k \in \{0, 1\}$: a level 1 VC from i to j exists, using two existing L_0 virtual channels from i to k and then from k to j:

(i) vc_{ij}^k can only be created if there are two L_0 VC connecting i and j:

$$2 \cdot L_{1vv_}\, vc_{ij}^k \leq \sum_{l \in V} L_{0_}\, vc_{ik}^l + \sum_{l \in V} L_{0_}\, vc_{kj}^l$$

(Because of C8-iii and C8-iv each of the sums on the right-hand side it at most 1).

(ii) The capacity of a VC is bounded by the capacity of the used VC:

$$L_{1vv_}\, vc_{ij}^k.capacity \leq \sum_{l \in V} L_{0_}\, vc_{ik}^l.capacity$$
$$L_{1vv_}\, vc_{ij}^k.capacity \leq \sum_{l \in V} L_{0_}\, vc_{kj}^l.capacity$$

(at most one summand on right-hand side in both inequalities is not equal to 0).

(iii) $L_{1vv_}\, vc_{ij}^k = 0 \Rightarrow L_{1vv_}\, vc_{ij}^k.capacity = 0$, i.e.,

$$L_{1vv_}\, vc_{ij}^k.capacity \leq L_{1vv_}\, vc_{ij}^k \cdot max_cap$$

At the end of this level we need a constraint for all cases, such that after every level at most one VC from i to j should exist:

$$\sum_{k \in V} \left(L_{0_}\, vc_{ij}^k + L_{1pv_}\, vc_{ij}^k + L_{1vp_}\, vc_{ij}^k + L_{1vv_}\, vc_{ij}^k \right) \leq 1$$

We now generalize level 1 to level $m \in [1, w-1]$.

(C5) Level m: (PC, VC). $L_{mpv_}\, vc_{ij}^k \in \{0, 1\}$: a level m VC from i to j exists, first using pc_{ik} and then an existing lower level virtual channel from k to j

(i) vc_{ij}^k can only be created if there is a PC and a VC connecting i and j:

$$2 \cdot L_{mpv_} vc_{ij}^k \leq pc_{ik} + \sum_{l \in V} \Bigg(L_{0_} vc_{kj}^l + \sum_{q=1}^{m-1} \left(L_{qpv_} vc_{kj}^l + L_{qvp_} vc_{kj}^l + L_{qvv_} vc_{kj}^l \right) \Bigg)$$

The used VC can be of every level from 0 to $m-1$ with an arbitrary intermediate node l:

- For L_0 we have only one case: $L_{0_} vc_{kj}^l$.
- For the other levels $q \in \{1, ..., m-1\}$ we have three cases: (PC, VC), (VC, PC) and (VC, VC).

(ii) The capacity of a VC is bounded by capacity of the used PC and VC:

$$L_{mpv_} vc_{ij}^k.capacity \leq pc_{ik}.capacity$$

$$L_{mpv_} vc_{ij}^k.capacity \leq \sum_{l \in V} \Bigg(L_{0_} vc_{kj}^l.capacity + \sum_{q=1}^{m-1} \left(L_{qpv_} vc_{kj}^l.capacity + L_{qvp_} vc_{kj}^l.capacity + L_{qvv_} vc_{kj}^l.capacity \right) \Bigg)$$

In every recursion step at most one summand in the outer sum on right-hand side in second inequality is not equal to 0.

(iii) $L_{mpv_} vc_{ij}^k = 0 \Rightarrow L_{mpv_} vc_{ij}^k.capacity = 0$, i.e.,

$$L_{mpv_} vc_{ij}^k.capacity \leq L_{mpv_} vc_{ij}^k \cdot max_cap$$

(C6) Level m: (VC, PC). Symmetric to the previous case:

$L_{mvp_} vc_{ij}^k \in \{0, 1\}$: a level m VC from i to j exists, first using an existing lower level virtual channel from i to k and then pc_{kj}

(i) vc_{ij}^k can only be created if there is a VC and a PC connecting i and j:

$$2 \cdot L_{mvp_} vc_{ij}^k \leq \sum_{l \in V} \Big(L_{0_} vc_{ik}^l + \sum_{q=1}^{m-1} \left(L_{qpv_} vc_{ik}^l + L_{qvp_} vc_{ik}^l + L_{qvv_} vc_{ik}^l \right) \Big) + pc_{kj}$$

(ii) The capacity of VC is bounded by the capacity of the used VC and PC:

$$L_{mvp_}vc_{ij}^{k}.capacity \leq \sum_{l\in V}\Bigg(L_{0_}vc_{ik}^{l}.capacity + \sum_{q=1}^{m-1}\big(L_{qpv_}vc_{ik}^{l}.capacity + L_{qvp_}vc_{ik}^{l}.capacity + L_{qvv_}vc_{kj}^{l}.capacity\big)\Bigg)$$

$$L_{mvp_}vc_{ij}^{k}.capacity \leq pc_{kj}.capacity$$

(iii) $L_{mvp_}vc_{ij}^{k} = 0 \Rightarrow L_{mvp_}vc_{ij}^{k}.capacity = 0$, i.e.,

$$L_{mvp_}vc_{ij}^{k}.capacity \leq L_{mvp_}vc_{ij}^{k} \cdot max_cap$$

(C7) Level m: (VC, VC). $L_{mvv_}vc_{ij}^{k} \in \{0,1\}$: a level m VC from i to j exists, using two existing lower level virtual channels from i to k and then from k to j

(i) vc_{ij}^{k} can only be created if there are 2 VC connecting i and j:

$$2 \cdot L_{mvv_}vc_{ij}^{k} \leq \sum_{l\in V}\Bigg(L_{0_}vc_{ik}^{l} + \sum_{q=1}^{m-1}\big(L_{qpv_}vc_{ik}^{l} + L_{qvp_}vc_{ik}^{l} + L_{qvv_}vc_{ik}^{l}\big)\Bigg) + \sum_{l\in V}\Bigg(L_{0_}vc_{kj}^{l} + \sum_{q=1}^{m-1}\big(L_{qpv_}vc_{kj}^{l} + L_{qvp_}vc_{kj}^{l} + L_{qvv_}vc_{kj}^{l}\big)\Bigg)$$

(ii) The capacity of VC is bounded by the capacity of the used VC:

$$L_{mvv_}vc_{ij}^{k}.capacity \leq \sum_{l\in V}\Bigg(L_{0_}vc_{ik}^{l}.capacity + \sum_{q=1}^{m-1}\big(L_{qpv_}vc_{ik}^{l}.capacity + L_{qvp_}vc_{ik}^{l}.capacity + L_{qvv_}vc_{kj}^{l}.capacity\big)\Bigg)$$

$$L_{mvv_}vc_{ij}^{k}.capacity \leq \sum_{l\in V}\Bigg(L_{0_}vc_{kj}^{l}.capacity + \sum_{q=1}^{m-1}\big(L_{qpv_}vc_{kj}^{l}.capacity + L_{qvp_}vc_{kj}^{l}.capacity + L_{qvv_}vc_{kj}^{l}.capacity\big)\Bigg)$$

(iii) $L_{mvv_}vc_{ij}^k = 0 \Rightarrow L_{mvv_}vc_{ij}^k.capacity = 0$, i.e.,

$$L_{mvv_}vc_{ij}^k.capacity \leq L_{mvv_}vc_{ij}^k \cdot max_cap$$

(C8) After every level at most one VC from i to j should exist:

$$\sum_{k \in V} \left(L_{0_}vc_{ij}^k + \sum_{q=1}^{m-1} (L_{qpv_}vc_{ij}^k + L_{qvp_}vc_{ij}^k + L_{qvv_}vc_{ij}^k) \right) \leq 1$$

Constraints for all levels of recursion.

(C9) $exist_vc_{ij}^k \in \{0,1\}$: 0, if VC does not exist, or 1, otherwise (which is also the number of VC from i to j via k in all levels). Let $exist_vc_{ij}^k := L_{0_}vc_{ij}^k + \sum_{q=1}^{n-1} \left(L_{qpv_}vc_{ij}^k + L_{qvp_}vc_{ij}^k + L_{qvv_}vc_{ij}^k \right)$. Then $exist_vc_{ij}^k \leq 1$.

(C10) A virtual channel can be used for a transaction if it exists

$$vc_{ij}^k(trans_{sr}) \leq exist_vc_{ij}^k$$

C.2 Example solution

To demonstrate our approach, we use a small example graph. We find an exact solution and also see the solution quality of the fast greedy algorithm. We will evaluate the performance of the greedy algorithm more systematically in Section 5, which is much faster than the ILP and can run on the Lightning Network. The small graph that is used in this example is shown in Figure 5 and consists out of two hub-like nodes H_1 and H_2, and four client nodes A, B, C and D. We say that each channel charges a base fee of 1, a proportional fee of 0.001 and has a capacity of 10k, distributed evenly among both users. The transactions that are executed have all value 10, there are three transactions from A to C, one transaction from A to B and one transaction from B to C. Further, we assume that H_1 is a malicious node. We chose this graph because it resembles the hub-and-spoke topology of the Lightning Network [42]. The transaction values and fees are chosen mainly for readability purposes: In the total fees below, the integer part of the number represents the money spent on the base fee, the fractional part of the money spent on the proportional fee.

In Figure 5, we show the virtual channels constructed by the algorithms in color, both solutions construct the same virtual channels. The rigorous algorithm builds a virtual channel VC_1 of capacity 40 between A and B, another virtual channel VC_2 of capacity 40 between B and C, and finally a third virtual channel VC_3 of capacity 40 between A and C. Finally, these virtual channels are used for routing the transactions. The total cost on fees is 3.11, which comes from three times the base fee of 1 (which is 3) and three times the proportional fee for capacity 40 (which is around .11) for creating the virtual channel. Since by then, all sender receiver pairs are directly connected, there are no additional routing fees.

In the fast algorithm, the same virtual channels are constructed. Since this algorithm greedily creates the best virtual channels for each sender-receiver pair

individually out of all transactions, the virtual channel VC_1 does not have enough capacity and has to be constructed twice. This incurs an extra base fee of value 1, totaling in 4.11 in fees.

Finally, if we look at the fees of routing these payments without any virtual channels, the total amount spent on fees is 11.11. Since we now need to route the payment from A to C three times over the path, the intermediaries charge a base fee each time, resulting in 9 coins alone. Further, in comparison to both the rigorous and fast algorithm, the payments without virtual channels are prone to value privacy attacks by H_1.

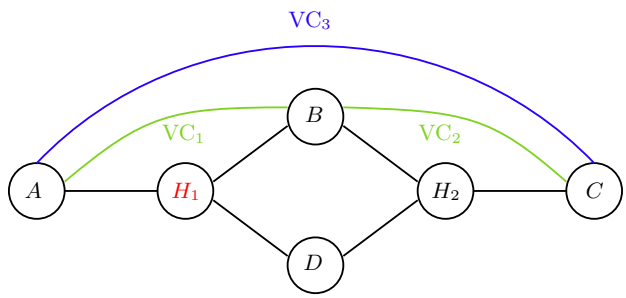

Fig. 5: Results of the rigorous algorithm run on a small sample graph. There are two hub nodes H_1 and H_2. Nodes A and C perform 3 transactions, A and B perform 1 transaction, and B and C perform 1 transaction. Node H_1 is corrupted. The resulting virtual channels that are constructed are VC_1 over $A - H_1 - B$, VC_2 over $B - H_2 - C$ and VC_3 over $A - B - C$.

C.3 Synergy among objectives: proof

We hereby remind Theorem 2 and present its proof.

Theorem 4 (Theorem 2, Section 4). *For any path p, it holds that $F(p) \supseteq V(p) \supseteq R(p)$ and $F(p) \supseteq V(p) \supseteq W(p)$.*

Proof. From the definitions of Algorithms 4 to 7, we observe the following. $V(p)$ is the set containing all corrupted nodes on the path p. Intuitively, if it is not, then there is a corrupted node left on the path which is not bypassed, thus value privacy attacks are not prevented. The two sets $R(p)$ and $W(p)$ do not have any honest nodes by definition (honest nodes do not need to be bypassed). $R(p)$ and $W(p)$ contain thus only malicious nodes, but they do not contain all the malicious nodes of path p. Consider for example path $s - c_1 - h_1 - c_2 - h_2 - c_3 - r$ (c_i representing corrupted and h_i honest nodes), where c_2 is in $V(p)$, but not in $R(p)$. It follows that $V(p) \supseteq R(p)$ and $V(p) \supseteq W(p)$.

It remains to show that $F(p) \supseteq V(p)$. For this, we merely observe that the set $F(p)$ contains every node on the path p, which includes every corrupted node, which is $V(p)$. □

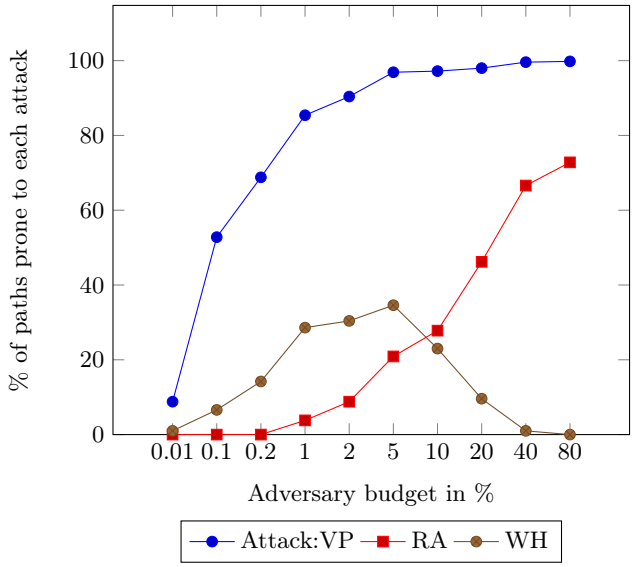

Fig. 6: Number of prone paths

D Computing the percentage of prone paths for different adversary budgets

We choose different adversary budgets ranging from 0.01% to 80% of the PCN capacity and check, how many of our payments are prone to the different attacks. We plot our results in Figure 6. We observe that value privacy attack is the cheapest, only 0.1% yields more than half of the payments being prone. Note that an adversary budget of 0.1% of the total capacity is still significant and requires in our data the staking of roughly 1.2 BTC. Achieving the wormhole attack is more expensive and the effectiveness peaks at around 5% (58.4 BTC in our data) as adversary budget, before going down again. This is due to the fact that this attack requires honest nodes in between. The most expensive attack to mount for our corruption strategy is relationship anonymity. Only after 10% adversary budget it affects more paths than wormhole attack.

Note that an adversary can use its corrupted nodes to carry out any or all of these attacks. Note that there might exist other strategies for corrupting nodes that are more effective for relationship anonymity and wormhole attacks (cf. [38]). From these results we choose 1, 2 and 5 percent as the adversary budget we want to investigate further, as with these budgets there are some (and not all) paths that are prone to each of the attacks and to be able to compare them more easily.

E Algorithms

In this section we give pseudocode definitions for the algorithms used to conduct our empirical evaluation. Algorithm 1 describes the algorithm for computing the cost ratio for preventing attacks. Algorithm 3 describes the algorithm for computing the cost ratio for the goal of optimizing fees. Algorithm 2 is the subprocedure to compute the corrutped nodes.

Further, we present the algorithms that return the sets of nodes that need to be bypassed to prevent value privacy attacks (Algorithm 4), relationship anonymity attacks (Algorithm 5), wormhole attacks (Algorithm 6) and to optimze fees (Algorithm 7).

Algorithm 1 Compute cost ratio of bypassing corrupted nodes

```
function COMPUTE_COST_RATIO(pcn, adv_budget, val_range, n, repetition)
    pcn_vc ← copy(pcn) //Copy the PCN graph for the VC simulation
    c ← compute_corrupted_nodes(pcn, adv_budget) //compute the most profitable corrupted nodes to a given adversary budget
    payments ← generate_payments(pcn, val_range, n, repetition) //generates n random payments with a random value in val_range in a PCN graph
    establish_cost_vc ← 0
    route_cost_vc ← 0
    unsuccessful_vc ← 0
    route_cost_pcn ← 0
    unsuccessful_pcn ← 0
    for (path, value, repetition) in payments do
        cost, new_path ← bypass_corrupted_nodes(c, path, value, repetition, pcn_vc)
        establish_cost_vc ← establish_cost_vc + cost
        for i ← 0; i < repetition ; i++ do
            success, fees_vc ← conduct_payment(pcn_vc, new_path, value) //executes the payment over the given path
            if success then
                route_cost_vc ← route_cost_vc + fees_vc
            else
                unsuccessful_pcn ← unsuccessful_pcn + 1
            success, fees_pcn ← conduct_payment(pcn, path, value) //executes the payment over the given path
            if success then
                route_cost_pcn ← route_cost_pcn + fees_pcn
            else
                unsuccessful_pcn ← unsuccessful_pcn + 1
    return (establish_cost_vc + route_cost_vc) / route_cost_pcn
```

Algorithm 2 Compute corrupted nodes

```
function COMPUTE_CORRUPTED_NODES(pcn, adv_budget)
    payments ← generate_payments(pcn, val_range, n, repetition) //generates n random payments
    node_map<node,int> ← {}
    for (path, value, repetition) in payments do
        for node in path do
            node_map[node] ← node_map[node]+1
    cost_benefit_map<node,int> ← {}
    for node in node_map.keys() do
        cost_benefit_map[node] ← (node_map[node]/n) / (cost(node)/adv_budget)
    corrupted_nodes ← []
    for (node,cost) in cost_benefit_map.sort_by_value(desc) do
        if adv_budget ≥ cost(node) then
            corrupted_nodes ← corrupted_nodes.append(node)
            adv_budget ← adv_budget - cost(node)
    return corrupted_nodes
```

Algorithm 3 Compute cost ratio for saving fees

```
function COMPUTE_COST_RATIO(pcn, val_range, n, repetition)
    pcn_vc ← copy(pcn) //Copy the PCN graph for the VC simulation
    payments ← generate_payments(pcn, val_range, n, repetition) //generates n random payments with a random value in val_range in a PCN graph
    establish_cost_vc ← 0
    route_cost_vc ← 0
    unsuccessful_vc ← 0
    route_cost_pcn ← 0
    unsuccessful_pcn ← 0
    for (path, value, repetition) in payments do
        cost, new_path ← open_profitable_vcs(c, path, value, repetition, pcn_vc)
        establish_cost_vc ← establish_cost_vc + cost
        for i ← 0; i < repetition ; i++ do
            success, fees_vc ← conduct_payment(pcn_vc, new_path, value) //executes the payment over the given path
            if success then
                route_cost_vc ← route_cost_vc + fees_vc
            else
                unsuccessful_pcn ← unsuccessful_pcn + 1
            success, fees_pcn ← conduct_payment(pcn, path, value) //executes the payment over the given path
            if success then
                route_cost_pcn ← route_cost_pcn + fees_pcn
            else
                unsuccessful_pcn ← unsuccessful_pcn + 1
    return (establish_cost_vc + route_cost_vc) / route_cost_pcn
```

Algorithm 4 Generating set of nodes to bypass for preventing VP

```
function COMPUTE_BYPASS_NODES_VP(path, corrupted_nodes)
    nodes_vp_path ← {}
    for node in path do
        if node in corrupted_nodes then
            nodes_vp_path ← nodes_vp_path ∪ {node}
    return nodes_vp_path
```

Algorithm 5 Generating set of nodes to bypass for preventing RA

```
function COMPUTE_BYPASS_NODES_RA(path, corrupted_nodes)
    Let nodes_ra_path_l be the set of connected nodes in corrupted_nodes adjacent to the sender path[0]
    Let nodes_ra_path_r be the set of connected nodes in corrupted_nodes adjacent to the receiver path[length(path)-1]
    if lenght(nodes_ra_path_l) < length(nodes_ra_path_r) then
        return nodes_ra_path_l
    return nodes_ra_path_r
```

Algorithm 6 Generating set of nodes to bypass for preventing WH

```
function COMPUTE_BYPASS_NODES_WH(path, corrupted_nodes)
    nodes_wh_path ← {}
    for while there exist honest nodes in path that is surrounded by corrupted nodes do
        Let hon_nodes be one of these honest nodes surrounded by corrupted nodes
        Let wh_l be the set of connected nodes in corrupted_nodes adjacent to the left of hon_nodes
        Let wh_r be the set of connected nodes in corrupted_nodes adjacent to the right of hon_nodes
        if lenght(wh_l) < length(wh_r) then
            Remove wh_l from path
            Add wh_l to nodes_wh_path
        else
            Remove wh_r from path
            Add wh_r to nodes_wh_path
        ▷ Note that one could also bypass hon_nodes, but we do not for simplicty here
    return nodes_wh_path
```

Algorithm 7 Generating set of nodes to bypass for optimizing fees

```
function OPTIMIZE_FEES(path, corrupted_nodes)
    nodes_fees_path ← path
    Remove first and last element from nodes_fees_path
    return nodes_fees_path
```

F Discussion

Fair establishment fee model for intermediaries. VCs are not used yet in practice and thus we do not know how fees are going to be charged. In Section 5, we assumed that the fee of opening a VC with some capacity α is the same as routing the payment of the same value through this path. Of course, other establishment fee models are possible.

First, in order to be fair to both the endpoints and intermediaries, VCs should have a limited lifespan. Remember that VCs require intermediaries to lock up some funds that they cannot use for other payments for which they would otherwise be able to charge fees. Note that this is different to PCs, where it is not problematic for two users to lock up their funds potentially indefinitely, as they do it only for their own funds. In other words, the fees that the intermediaries receive have to be proportionate to the time for which the money is locked up, i.e., the lifespan of a VC. Since channel capacities are finite, this implies that the lifespan is also finite.

The way we previously modelled the fees is that VC creation fees are independent of the lifespan of a channel. In fact, one could argue that the fees for routing payments should then also be dependent on the collateral timeout, which in practice it is not. However, it is hard to model this in our experiments, as due to the nature of the PCN, it is unknown how many payments are processed in which time frames. For this reason, we chose our simplified fee model.

In order for VCs to still be profitable in this fairer fee model, one would have to find a number k of payment repetitions (for each of our n payments), a lifetime of the VC t, the base fee f, the fee p proportional to the amount and a fee v that is proportional to the lifetime and the capacity of the VC, such that the inequality $\sum_n(\sum_k(f + \alpha \cdot p)) \geq \sum_n(f + k \cdot \alpha \cdot p + k \cdot \alpha \cdot v \cdot t)$ holds.

This simplifies to the following: $(k - 1) \cdot f \geq k \cdot \alpha \cdot v \cdot t$. We note that this inequality is not exact, as the forwarded amount gets smaller when intermediaries already deduct fees. Nonetheless, the investigation of this or other fee models is interesting future work.

Utilizing bidirectionality of VCs. The greedy algorithms create virtual channels with transactions in one direction in mind. This means, that on a path where several transactions are executed the capacity is chosen as the sum of the amount of these transactions. In reality, the capacity can be lower, if there are transaction in the other direction in the same time frame. Since we do not model any timing as mentioned above, we also do not capture this.

A practical scenario where this is useful might be where two payment providers route a substantial amount of payments through a hub back and forth. Note that the sum of all their payments can be very large, but since they send it back and forth the capacity of the virtual channel can be smaller.

Blocking capacity with VC. If a VC is used only sparsely, then it might block the capacity of the underlying payment channels.

VC routing fee. Similar to the establishment fee, it is unclear what fee the users of a virtual channel will charge in practice for letting other users route

their payments through the virtual channel. In this work, we assume this fee to be the same as fees that are charged for one of the underlying channels.

Privacy of VC creation. Following our strategies, we aim to prevent the on-chain privacy attacks on value privacy and relationship anonymity for payments. However, since opening or closing a virtual channel is an operation in which potentially corrupted intermediaries participate, some information might be leaked while opening or closing a virtual channel.

We assume that VCs used in the same way as payment channels, e.g., for routing payments through them. To make other users aware of accepting payment being routed through channels, channels need to be announced publicly. Therefore, we disregard privacy concerns for VC opening/closing and instead assume that endpoints and capacity of VCs are announced publicly, thereby already leaking the information on endpoints and capacity.

Using VCs for routing. In our heuristic evaluation, we consider the payments that are repeated once or multiple times and conclude that if payments are conducted more than once along a path, opening a direct VC is the best strategy. However, for payments that are conducted only once, the VCs that were created in this fashion can be used for having shorter paths, which means having fewer fees and being less at risk for attacks due to having fewer intermediaries.